

Teoria Współbieżności

Laboratorium 4

Producenci i konsumenci z losową ilością pobieranych i wstawianych porcji

Kacper Bieniasz

28 października 2024

1 Dane techniczne sprzętu

Obliczenia zostały wykonane na komputerze o następującej specyfikacji:

- Procesor: AMD Ryzen 7 5800U
- Pamięć RAM: 16 GB DDR4 3200 MHz (2×8GB)
- System operacyjny: Windows 11 Home x64

2 Treść zadań

2.1 Treść zadania

Zadanie rozszerza problem producentów i konsumentów. Polecenie przedstawione w punktach:

- Bufor o rozmiarze $2M$
- Jest m producentów i n konsumentów
- Producent wstawia do bufora losową liczbę elementów (nie więcej niż M)
- Konsument pobiera losową liczbę elementów (nie więcej niż M)
- Zaimplementować przy pomocy monitorów Javy oraz mechanizmów Java Concurrency Utilities
- Przeprowadzić porównanie wydajności (np. czas wykonywania) vs. różne parametry, zrobić wykresy i je skomentować.

3 Implementacja zadania

Biblioteki zaimportowane na potrzeby programu:

```
1 import java.io.FileWriter;  
2 import java.io.IOException;  
3 import java.util.concurrent.ExecutorService;  
4 import java.util.concurrent.Executors;  
5 import java.util.Random;  
6 import java.util.concurrent.locks.Condition;  
7 import java.util.concurrent.locks.Lock;  
8 import java.util.concurrent.locks.ReentrantLock;
```

Kod Java 1: Zaimportowane biblioteki

Kod (2) zawiera klasę **Main**, której zadaniem jest tworzenie producentów i konsumentów w zależności od określonych parametrów. Program był testowany dla $M = \{100, 500, 1000, 5000\}$ oraz liczby producentów i konsumentów $m, n \in \{1, \dots, 39\}$. Tutaj również odbywa się pomiar czasu oraz zapisanie statystyk wykonania programu w celu sporządzenia wykresu.

```

1 public class Main {
2     public static void main(String[] args) {
3         int[] maxPortions = {100, 500, 1000, 5000};
4         int[] numProducersList = new int[20];
5         int[] numConsumersList = new int[20];
6         int step = 50 / (19);
7         for (int i = 0 ; i<numProducersList.length; i++) {
8             numProducersList[i] = 1 + i * step;
9         }
10        for (int i = 0 ; i<numConsumersList.length; i++) {
11            numConsumersList[i] = 1 + i * step;
12        }
13        try (FileWriter writer = new FileWriter("performance_results.csv")) {
14            writer.write("MaxPortion,NumProducers,NumConsumers,ExecutionTimeMs\n");
15            for (int maxPortion : maxPortions) {
16                for (int numProducers : numProducersList) {
17                    for (int numConsumers : numConsumersList) {
18                        int bufferCapacity = 2 * maxPortion;
19                        Buffer buffer = new Buffer(bufferCapacity);
20                        ExecutorService executor = Executors.newFixedThreadPool(numProducers + numConsumers);
21                        long startTime = System.nanoTime();
22                        for (int i = 0; i < numProducers; i++) {
23                            executor.execute(new Producer(buffer, maxPortion));
24                        }
25                        for (int i = 0; i < numConsumers; i++) {
26                            executor.execute(new Consumer(buffer, maxPortion));
27                        }
28                        executor.shutdown();
29                        long endTime = System.nanoTime();
30                        long executionTimeMs = (endTime - startTime) / 1_000_000;
31                        writer.write(maxPortion + "," + numProducers + "," + numConsumers + "," + executionTimeMs + "\n");
32                        System.out.println("Configuration: M=" + maxPortion + ", Producers=" + numProducers + ", Consumers=" + numConsumers + "-> Time: " + executionTimeMs + " ms");
33                    }
34                }
35            }
36        } catch (IOException e) {
37            e.printStackTrace();
38        }
39    }
40 }
41 }

```

Kod Java 2: Klasa **Main**

Działanie klasy **Buffer** przedstawia kod (3). Jej zadaniem jest stworzenie miejsca na przechowywanie produktów. Zajmuje się ich umieszczeniem i usunięciem. Posiada również mechanizmy do zapewnienia synchronizacji. Obie funkcje do dodawania i usuwania są zmodyfikowane do obsługi wielu produktów.

```

1 class Buffer {
2     private final int[] buffer;
3     private int size = 0;
4     private final int maxSize;
5     private final Lock lock = new ReentrantLock();
6     private final Condition notFull = lock.newCondition();
7     private final Condition notEmpty = lock.newCondition();
8     public Buffer(int maxSize) {
9         this.maxSize = maxSize;
10        buffer = new int[maxSize];

```

```

11     }
12     public void produce(int count) throws InterruptedException {
13         lock.lock();
14         try {
15             while (size + count > maxSize) {
16                 notFull.await();
17             }
18             for (int i = 0; i < count; i++) {
19                 buffer[size++] = 1;
20             }
21             notEmpty.signalAll();
22         } finally {
23             lock.unlock();
24         }
25     }
26     public void consume(int count) throws InterruptedException {
27         lock.lock();
28         try {
29             while (size < count) {
30                 notEmpty.await();
31             }
32             for (int i = 0; i < count; i++) {
33                 buffer[--size] = 0;
34             }
35             notFull.signalAll();
36         } finally {
37             lock.unlock();
38         }
39     }
40 }

```

Kod Java 3: Klasa **Buffer**

Klasy **Producer** oraz **Consumer** przedstawia kod (4). Obie klasy wyglądają podobnie jak w poprzednich zadaniach. Są jednak zmodyfikowane o produkcję i konsumpcję o losową wartość nie większą od pojemności bufora.

```

1 class Producer implements Runnable {
2     private final Buffer buffer;
3     private final int maxPortion;
4     private final Random random = new Random();
5     public Producer(Buffer buffer, int maxPortion) {
6         this.buffer = buffer;
7         this.maxPortion = maxPortion;
8     }
9     @Override
10    public void run() {
11        try {
12            while (true) {
13                int count = random.nextInt(maxPortion) + 1;
14                buffer.produce(count);
15                Thread.sleep(random.nextInt(100));
16            }
17        } catch (InterruptedException e) {
18            Thread.currentThread().interrupt();
19        }
20    }
21 }
22 class Consumer implements Runnable {
23     private final Buffer buffer;
24     private final int maxPortion;
25     private final Random random = new Random();
26 }

```

```

27 public Consumer(Buffer buffer, int maxPortion) {
28     this.buffer = buffer;
29     this.maxPortion = maxPortion;
30 }
31 @Override
32 public void run() {
33     try {
34         while (true) {
35             int count = random.nextInt(maxPortion) + 1;
36             buffer.consume(count);
37             Thread.sleep(random.nextInt(100));
38         }
39     } catch (InterruptedException e) {
40         Thread.currentThread().interrupt();
41     }
42 }
43 }

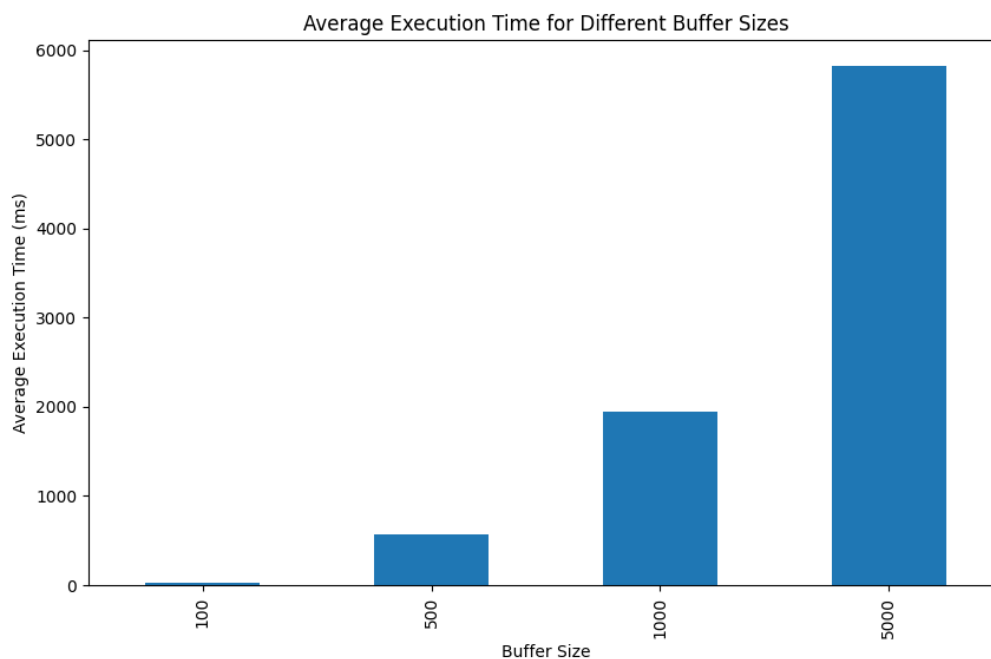
```

Kod Java 4: Klasy **Producer** i **Consumer**

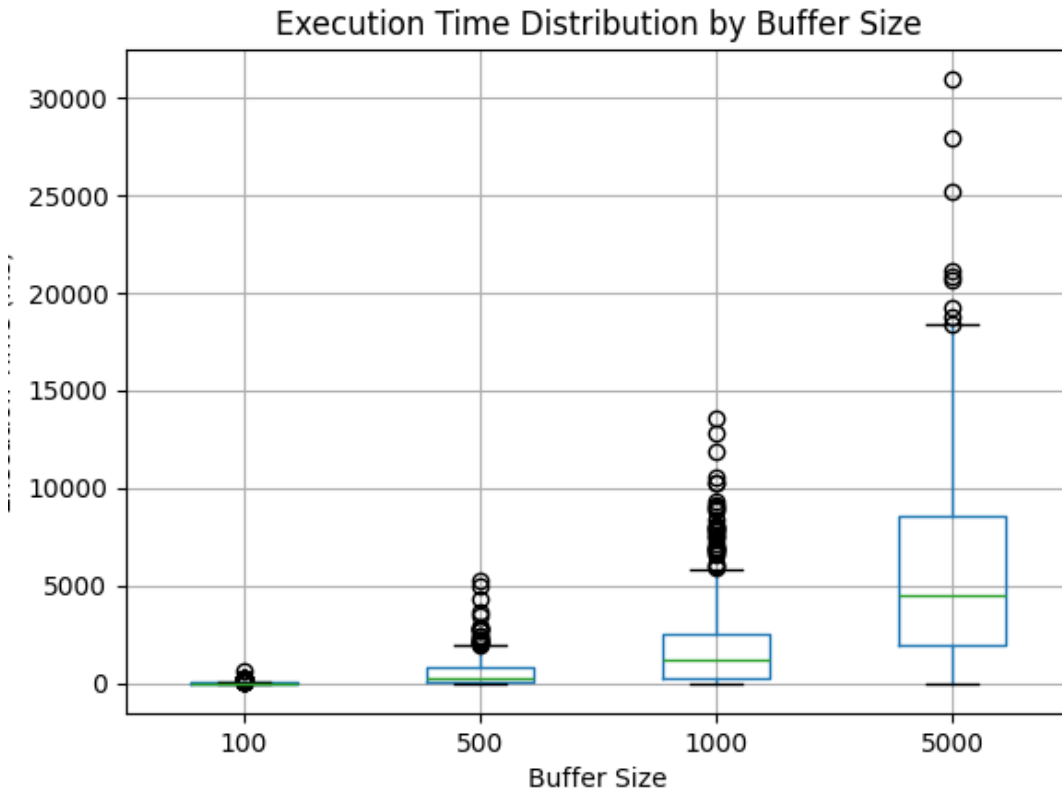
4 Wyniki dla różnych parametrów programu

4.1 Wykresy wspólne dla różnych wielkości bufora

Wykres (1) słupkowy przedstawia średni czas wykonania programu dla danego bufora, natomiast wykres (2) pudełkowy pokazuje rozkład czasu wykonania w zależności od wielkości bufora. Dostarcza on więcej informacji pokazując między innymi wartości odstające jak i kwantyle. W obu przypadkach możemy dostrzec, że zwiększanie pojemności bufora powoduje zwiększenie czasu wykonania.



Rysunek 1: Wykres słupkowy czasu wykonania w zależności od rozmiaru bufora

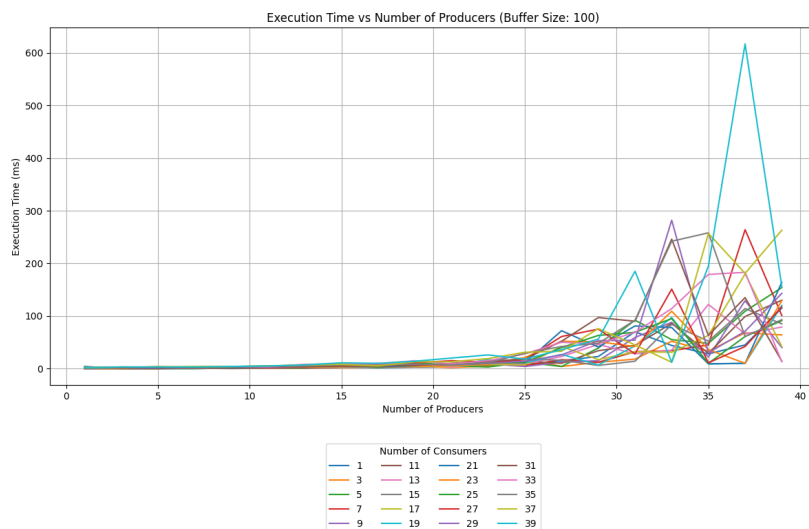


Rysunek 2: Wykres pudełkowy czasu wykonania w zależności od rozmiaru bufora

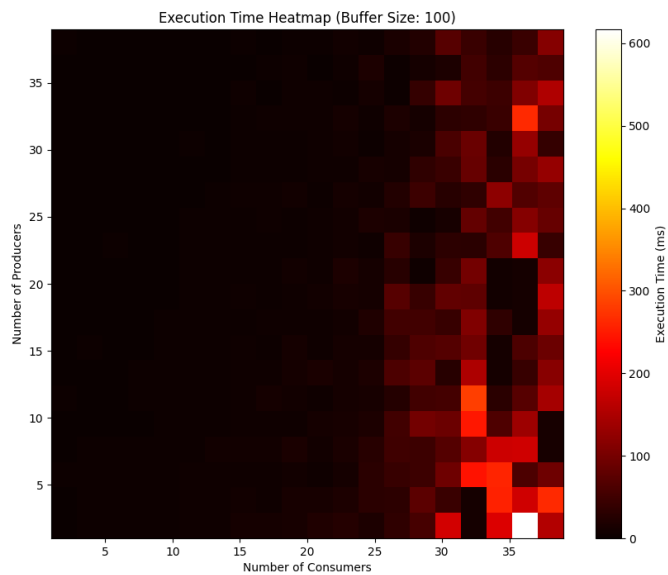
4.2 Wykresy dla różnych rozmiarów bufora

Wykres liniowy pokazuje czas wykonania w zależności od liczby producentów, każda krzywa przedstawia inną liczbę konsumentów. Lepiej zachowanie programu obrazuje heat mapa czyli wykres, gdzie na podstawie liczby konsumentów i producentów jasność danej komórki odwzorowuje czas wykonania. Działanie naszego programu można przedstawić jako funkcję dwóch zmiennych. Kolejnym sposobem na przedstawienie zachowania programu będzie wykres płaszczyzny, gdzie na osiach X oraz Y będą liczby producentów i konsumentów, zaś oś Z reprezentuje czas wykonania. Każdy rozmiar bufora posiada omawiane powyżej wykresy.

Bufor o rozmiarze 100

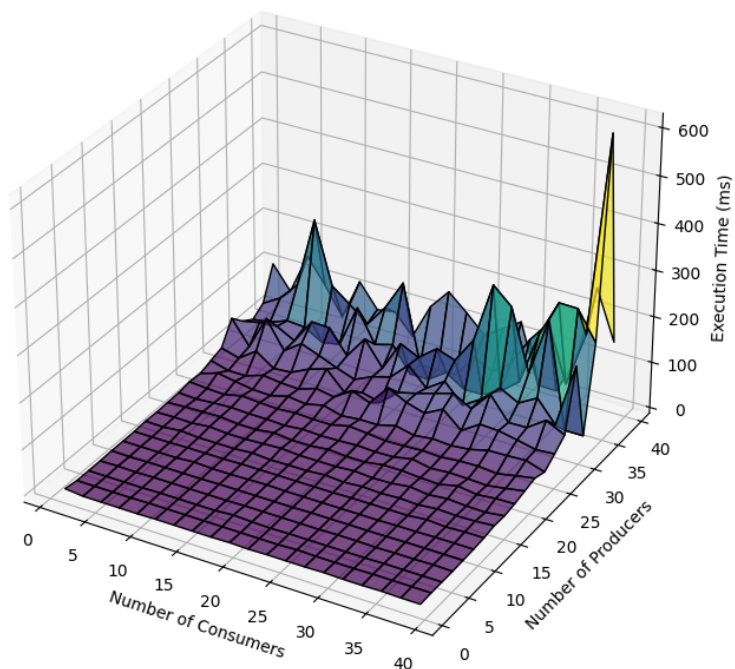


Rysunek 3: Wykres liniowy czasu wykonania w zależności od liczby konsumentów i producentów dla bufora o rozmiarze 100



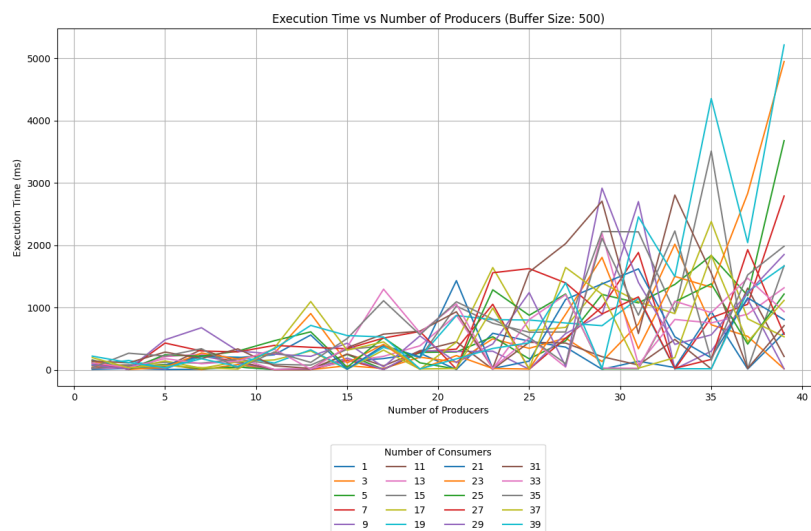
Rysunek 4: Wykres heat mapy czasu wykonania w zależności od liczby konsumentów i producentów dla bufora o rozmiarze 100

Execution Time Surface for Buffer Size 100

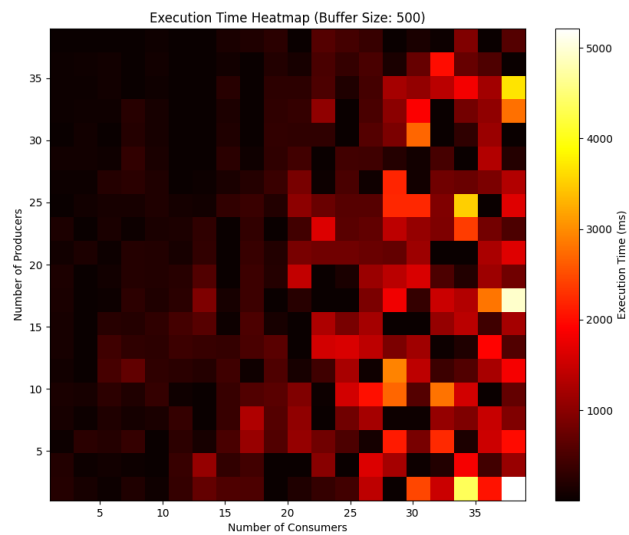


Rysunek 5: Wykres płaszczyzny czasu wykonania w zależności od liczby konsumentów i producentów dla bufora o rozmiarze 100

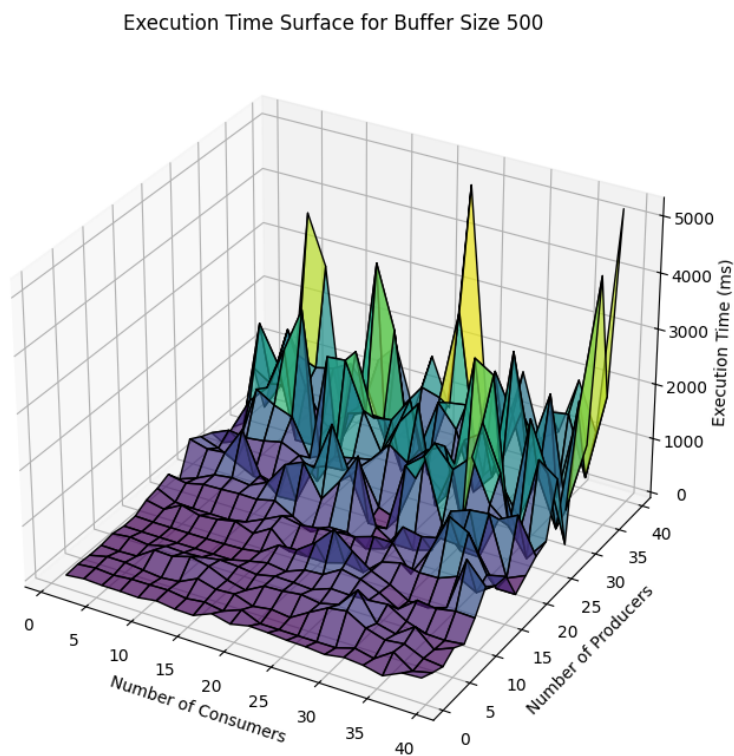
Bufor o rozmiarze 500



Rysunek 6: Wykres liniowy czasu wykonania w zależności od liczby konsumentów i producentów dla bufora o rozmiarze 500

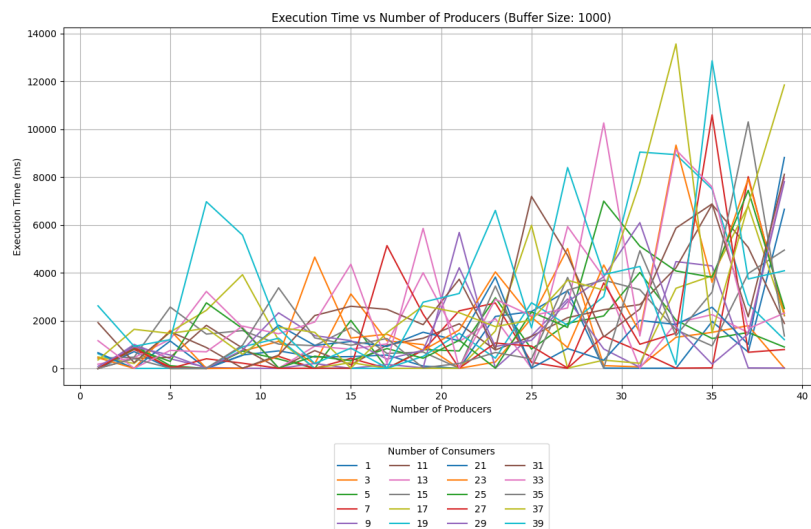


Rysunek 7: Wykres heat mapy czasu wykonania w zależności od liczby konsumentów i producentów dla bufora o rozmiarze 500

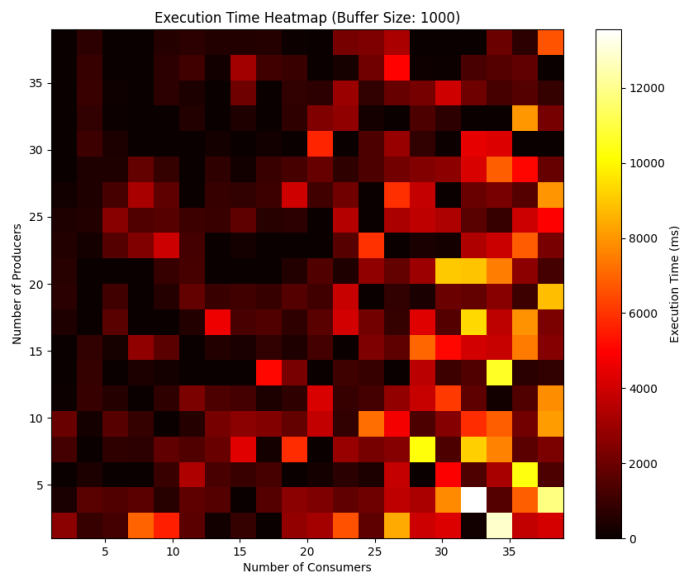


Rysunek 8: Wykres płaszczyzny czasu wykonania w zależności od liczby konsumentów i producentów dla bufora o rozmiarze 500

Bufor o rozmiarze 1000

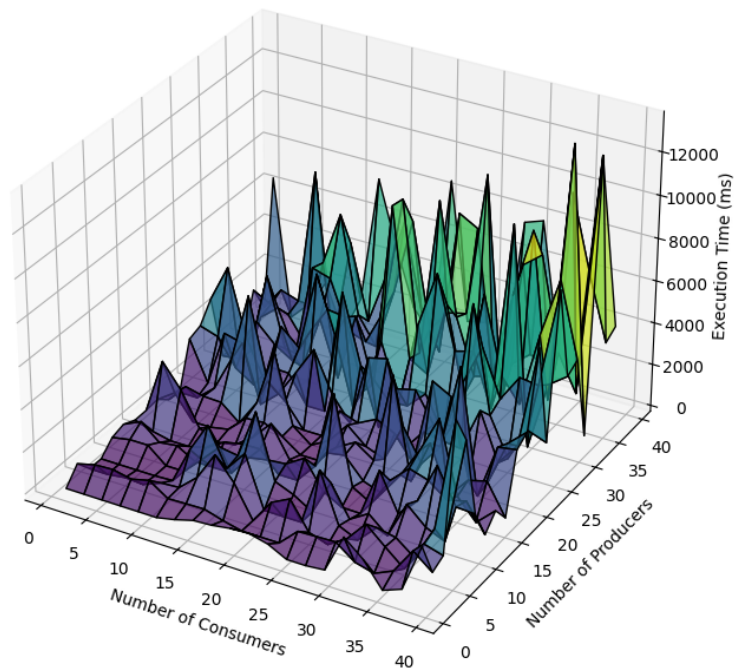


Rysunek 9: Wykres liniowy czasu wykonania w zależności od liczby konsumentów i producentów dla bufora o rozmiarze 1000



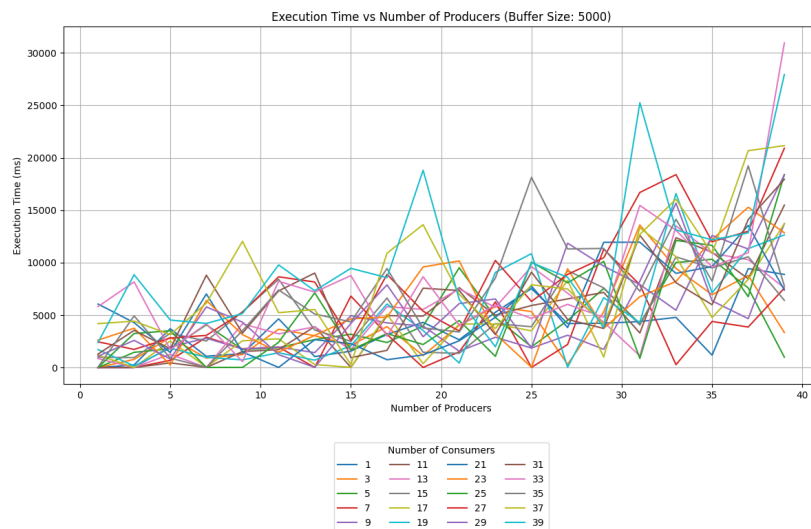
Rysunek 10: Wykres heat mapy czasu wykonania w zależności od liczby konsumentów i producentów dla bufora o rozmiarze 1000

Execution Time Surface for Buffer Size 1000

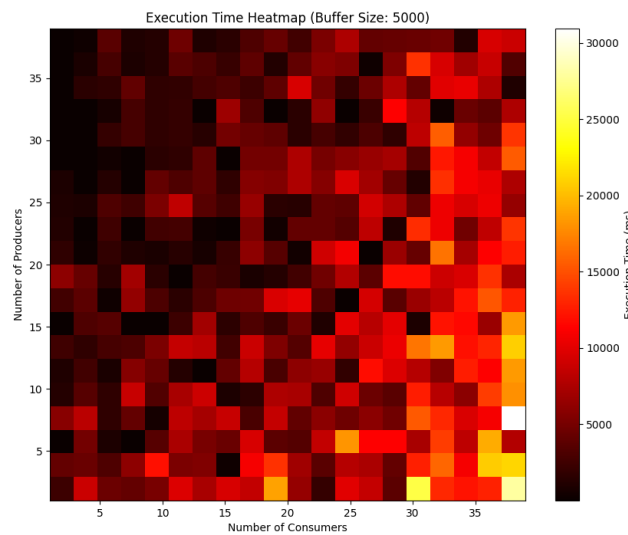


Rysunek 11: Wykres płaszczyzny czasu wykonania w zależności od liczby konsumentów i producentów dla bufora o rozmiarze 1000

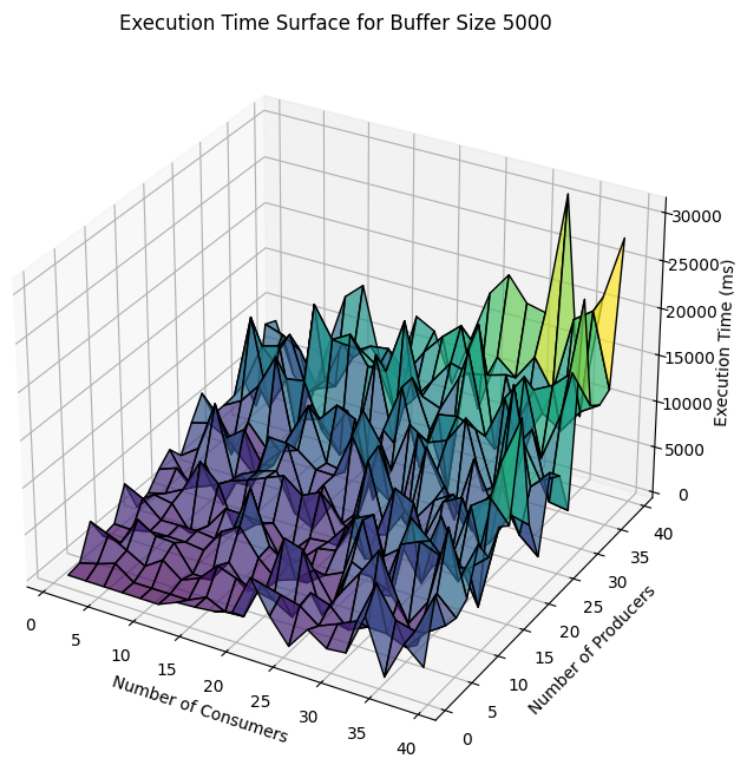
Bufor o rozmiarze 5000



Rysunek 12: Wykres liniowy czasu wykonania w zależności od liczby konsumentów i producentów dla bufora o rozmiarze 5000



Rysunek 13: Wykres heat mapy czasu wykonania w zależności od liczby konsumentów i producentów dla bufora o rozmiarze 5000



Rysunek 14: Wykres płaszczyzny czasu wykonania w zależności od liczby konsumentów i producentów dla bufora o rozmiarze 5000

5 Wnioski

Z analizy wyników można wnioskować, że wraz ze wzrostem rozmiaru bufora oraz liczby producentów i konsumentów, czas wykonania operacji rośnie. Dla mniejszych buforów, takich jak 100, czas wykonania jest relatywnie niski i stabilny, co sugeruje, że przy niskim obciążeniu i mniejszym rozmiarze bufora system działa efektywnie. Wraz ze wzrostem rozmiaru bufora do wartości takich jak 1000 i 5000, czasy wykonania stają się coraz bardziej zróżnicowane, a przy dużej liczbie producentów i konsumentów osiągają wyższe wartości, co oznacza, że system staje się mniej przewidywalny i mniej wydajny przy wysokim obciążeniu. Heatmapy i wykresy powierzchniowe jasno pokazują, że największe czasy wykonania występują, gdy zarówno liczba producentów, jak i konsumentów jest duża, co prowadzi do intensywnego współdzielenia zasobów i zwiększenia rywalizacji o dostęp do bufora. Z kolei wykresy pudełkowe ukazują, że większe bufora mają bardziej zróżnicowane rozkłady czasów wykonania, z wieloma wartościami odstającymi, co sugeruje, że w systemach o dużym obciążeniu większy bufor może nie gwarantować lepszej wydajności. Ogólnie rzecz biorąc, program pokazuje, że optymalna wydajność jest osiągana przy umiarkowanej liczbie producentów i konsumentów oraz przy stosunkowo mniejszych rozmiarach bufora. Zwiększanie rozmiaru bufora oraz liczby wątków prowadzi do wzrostu czasu wykonania i większego rozrzutu wyników, co może być problematyczne w aplikacjach wymagających stabilności i przewidywalności.

Literatura

- [1] Per Brinch Hansen: Java Insecure Parallelism. ACM SIGPLAN Notices, April 1999.
- [2] Thomas W. Christopher, George K. Thiruvathukal: High-Performance Java Platform Computing, Prentice Hall, Luty 2001.
- [3] <https://docs.oracle.com/javase/1.5.0/docs/guide/concurrency/overview.html>
- [4] <https://jenkov.com/tutorials/java-util-concurrent/index.html>
- [5] <https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/locks/Condition.html>